

Machine Learning and Deep Learning

Purna Chandra Nandigana



30.09.2026

What problem am I solving?

- **Problem:** flag every anomalous timestep in 18-sensor recordings of a batch-distillation plant (1 Hz).
- **Challenge:** the 28 training runs have *no labels*.
 - 10 validation runs are labelled (24% anomalous)
 - 53 test runs are scored on the Codabench portal (binary F1)
- **Approach:** learn what *normal* looks like, then flag departures from it (novelty detection).

Key Takeaway: No anomaly labels for training, so the model must learn normal behaviour.

What does the data look like?

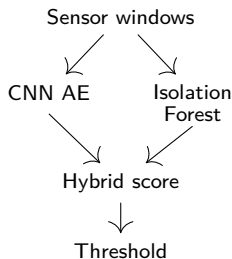
Split	Runs	Timesteps	Labels
Train	28	189,444	none
Validation	10	79,688	per timestep
Test	53	316,024	hidden (portal)

- **Sensors:** temperatures, flows, pressures and levels
- **Anomaly rate per validation run:** from 0.8% to 81%
- **No separate test labels:** the 10 labelled runs are used for tuning *and* for every local metric

Key Takeaway: Very few labelled runs, and they differ a lot from each other.

Phase 1: Where did I start?

- **Baseline:** Isolation Forest on 216 tabular features
→ val. F1 = 0.437
- **First deep model:** hybrid of a shallow CNN autoencoder and a window Isolation Forest (weights 0.85 / 0.15)
- Its reported F1 ≈ 0.62 was measured on *window-level* labels and could not be reproduced



Key Takeaway: The starting point was a hybrid model whose numbers could not yet be trusted.

Phase 1: What did I have to fix first?

- **Six correctness bugs** found while rebuilding the notebooks into one pipeline:
 - windows crossed run boundaries
 - threshold taken off by one
 - Isolation Forest features flattened wrongly
 - scoring per window instead of per timestep
 - training reverted from 50 to 10 epochs
 - fusion weight hand-set instead of tuned
- **Result after fixing:** hybrid val. $F1 = 0.494$
- **Portal:** 0.399 (public) / 0.471 (private)

Key Takeaway: Correct evaluation first; only then do the numbers mean anything.

Phase 1: How did the autoencoder improve?

Round	CNN F1	CNN ROC-AUC
1: corrected pipeline	0.408	0.574
2: checkpoint selection	0.458	0.614
3: dilated architecture	0.484	0.605

- **Checkpoint selection:** keep the best validation epoch, not the last
- **Dilated architecture:** wider view of time (61 timesteps)
- **Problem left:** ROC-AUC stayed near 0.6 and the threshold flagged up to 89% of timesteps

Key Takeaway: F1 went up, but ranking quality did not: the score itself was the bottleneck.

Phase 2: How does the final pipeline work?

Sensor runs $\rightarrow$ standard scaling (train only)



Per-run windows (200×18)



Dilated CNN autoencoder



Per-sensor reconstruction error



Mahalanobis score $\rightarrow$ one score per timestep



$F_{0.5}$ threshold (validation)



Final prediction (0 / 1)

Key Takeaway: Same autoencoder as Phase 1, but a new score and a new threshold.

Phase 2: How does the CNN autoencoder learn normal behaviour?

- **Input:** window of 200 timesteps
× 18 sensors
- **Encoder:** 4 dilated residual blocks (dilation 1, 2, 4, 8) → MaxPool
- **Decoder:** ConvTranspose1d → 2 residual blocks → Conv1d
- **Training:** MSE, Adam (lr 10^{-3}), batch 64, 50 epochs, best checkpoint every 5 epochs

Model Flow

- Input 18×200
- Dilated blocks (32, 32, 32, 64)
- MaxPool → 64×100
- ConvTranspose1d
- Dilated blocks (32, 32)
- Output 18×200

Key Takeaway: 59,858 parameters; the bottleneck makes it good only at normal data.

Phase 2: How is the anomaly score calculated?

- **Before:** one flat error averaged over all sensors, so a spike on one precise sensor was diluted
- **Now:** one error per sensor, $e_w \in \mathbb{R}^{18}$

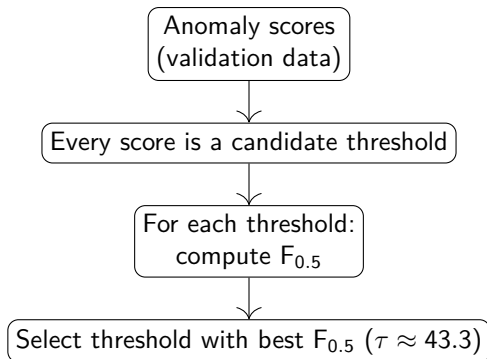
Per-Sensor Mahalanobis Score

$$s(w) = (e_w - \mu)^\top \Sigma^{-1} (e_w - \mu)$$

- μ, Σ : fitted on normal validation windows (Ledoit–Wolf)
- **Per timestep:** mean score of all windows that contain it

Key Takeaway: Each sensor is judged against its own normal error level.

Phase 2: How is the decision boundary selected?



Key Idea: $F_{0.5}$ weights precision above recall, which stops the model from flagging most of the data.

Phase 2: Did the new score help?

Metric (validation)	Round 3	Round 4
F1	0.484	0.581
Precision / recall	0.347 / 0.796	0.601 / 0.562
ROC-AUC	0.605	0.768
Flagged (true 0.24)	0.894	0.224

- **ROC-AUC** +0.16: better ranking, independent of the threshold
- **Portal**: 0.471 → **0.59**

Key Takeaway: Changing the score, not the network, gave the largest gain of the project.

Phase 2: Did fusion with LOF help?

- **Test:** fuse CNN with a Local Outlier Factor detector (contributed by my teammate)
- **Weight** chosen by 5-fold cross-validation grouped by run

Val. F1	CNN only	LOF only	Hybrid
raw features	0.592	0.461	0.549
standardised	0.568	0.481	0.568

- 9 of 10 folds chose the CNN alone (LOF weight 0.03 / 0.00)

Key Takeaway: Fusion was rejected; the final model is the CNN autoencoder alone.

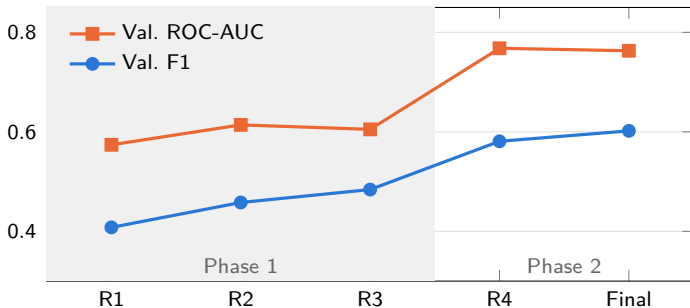
Final Performance Comparison (Test Set)

Model	Val. F1	Portal F1 (Test)
Phase 1: hybrid	0.494	0.399 / 0.471
Phase 2: Mahalanobis score	0.581	0.59
Phase 2: CNN only (final)	0.602	0.63

- Final model: val. ROC-AUC 0.763
- 0.59 $\rightarrow$ 0.63: a new training run of the same method (run-to-run spread about ± 0.02)

Key Insight: The CNN autoencoder with Mahalanobis scoring achieves the best result, portal F1 = 0.63.

How did performance change across the phases?



Key Takeaway: Phase 1 raised F1 slowly; Phase 2 moved the ranking quality itself.

How reliable is the final result?

- **Tested after the final submission** (none adopted):
 - F1-optimal threshold: $+0.024$ val. F1, but flags 84% of test
 - Per-run normalisation: ROC-AUC $0.769 \rightarrow 0.677$
 - Normal model fitted on train: ROC-AUC 0.611
- **Leave-one-run-out:** validation is optimistic by 0.070 ROC-AUC and 0.058 F1
- **Val. vs test:** 24% vs 62% of timesteps flagged at the same threshold

Key Takeaway: The method held up, but validation numbers should be read as optimistic.

What are the key parameters of the final system?

Component	Parameter Value
Windowing	Window size = 200, train stride = 10
CNN Autoencoder	Dilations = 1, 2, 4, 8
CNN Autoencoder	Filters = 32, 32, 32, 64
CNN Autoencoder	Batch size = 64
CNN Autoencoder	Learning rate = 1e-3, 50 epochs
Scoring	Mahalanobis, Ledoit–Wolf covariance
Threshold	$F_{0.5}$, $\tau \approx 43.3$

Key Insight: Performance depended most on how the error is scored and thresholded.

Summary & Next Steps

- **Summary:** dilated CNN autoencoder with per-sensor Mahalanobis scoring and an $F_{0.5}$ threshold.
- **Result:** portal F1 = **0.63** (val. F1 0.602, ROC-AUC 0.763).
- **Key Outcome:** Phase 2's new score lifted ROC-AUC from 0.605 to 0.768.
- **Next Steps:**
 - Leave-one-run-out validation for every tuning step
 - Confirm how the portal computes F1
 - Average over several training seeds

Final Message: How the model's error is scored mattered more than model size.